

Sovereign AI-Powered IDS/SIEM

Consolidated Implementation Plan — Revision 2

Synthesized from the Gemini research report, the Qwen technical plan, and an independent third-party technical review, fact-checked against current (August 2026) licensing and hosting status.

1. Executive Summary & What Changed

This revision folds in a third review that checked the first version of this plan against actual security-engineering practice. Three corrections and four additions came out of that check — all verified before being applied here.

Corrections to this document:

- TheHive is more nuanced than “no longer free.” A restricted Community license does exist (self-hosted, but limited to discovery/testing/training/education, and it requires requesting a license key). DFIR-IRIS remains the default recommendation for anything beyond that scope.
- TON_IoT (one of the three benchmark datasets) is free for academic research only — commercial use requires permission from UNSW, the dataset's author.
- The original architecture had the SOC-copilot LLM generating XAI/SHAP explanations. That's backwards and risks a hallucinated explanation for why an alert fired. SHAP has to come from the trained tabular model against the real feature vector; the LLM's only job is translating that into plain language.

Additions to this document:

- A dedicated ML evaluation methodology (Section 4) — proper data splitting, calibration, and metrics that actually reflect real-world performance rather than inflated benchmark accuracy.
- ML alerts now explicitly run in shadow mode — visible, but not changing alert severity — until validated, not just gated before automated blocking.
- Kafka and Flink are deferred until a measured need justifies them; the plan now starts with Wazuh's own APIs and a lightweight Python/ONNX scorer.
- A new guardrails section treats the detection stack itself as attack surface: poisoned training labels, adversarial flows, and prompt injection via log fields reaching the LLM.

A note on scope: this revision assumes you're building this for personal/lab use rather than production deployment protecting real infrastructure. The safety-critical disciplines (shadow mode, human-approval gating) stay regardless of scale — they're what stops a false positive from becoming a real outage. The heavier formal governance layer (SBOM tracking, signed model artifacts, monthly compliance review, a written threat-model document) is mentioned where relevant but kept optional. If this is heading toward protecting something real, say so and that layer should move from optional to required.

2. Corrected Reference Architecture

The diagram below reflects the corrected design: telemetry flows into a single Wazuh hub, the ML scorer runs in shadow mode and produces its own SHAP explanation (never the LLM), and every response — automated or investigative — passes through a mandatory human approval gate. The scale-out layer (Kafka, Flink, ClickHouse, Grafana, Keep, CrowdSec) is deliberately drawn as optional — add it only once Phases 1–6 prove insufficient on their own.

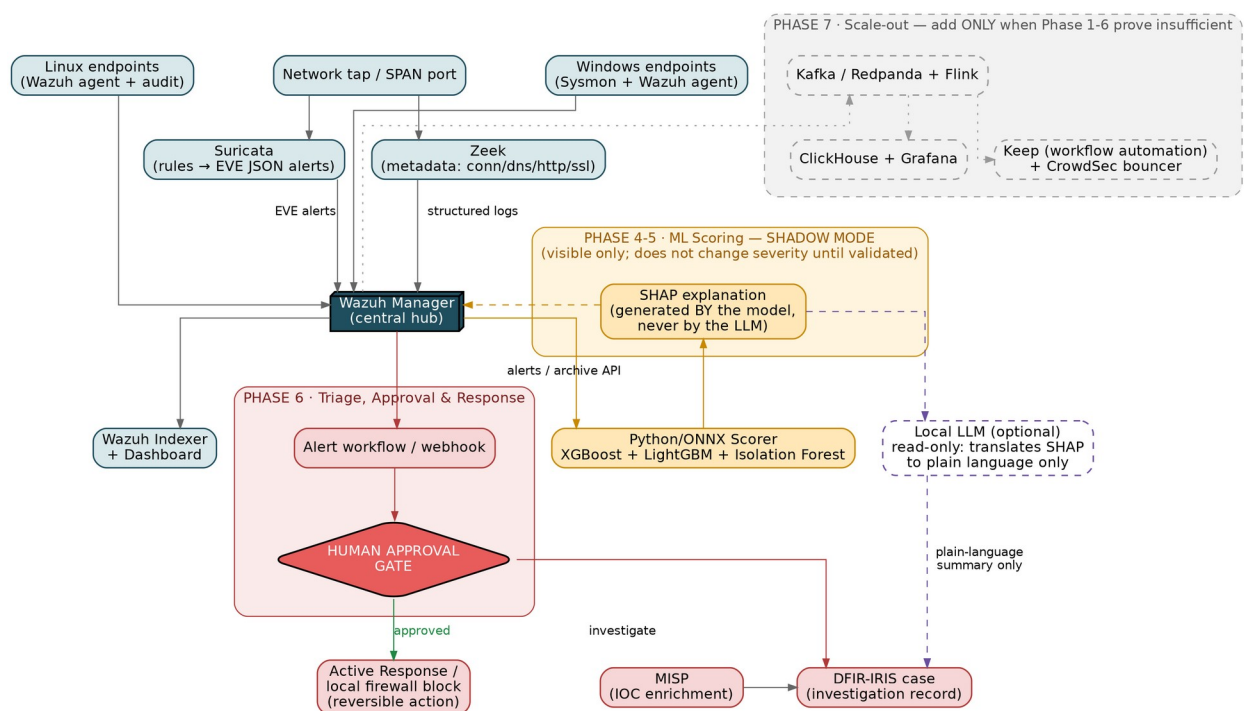


Figure 1. Corrected detection-to-response flow. Dashed elements are deferred until measured need.

Modern Wazuh ships its own Wazuh Indexer, an Apache-2.0 OpenSearch fork — not raw Elasticsearch. Same RAM realities as Elasticsearch (~16GB comfortable minimum for the indexer alone at real event volume), but no separate license concern.

3. Final Tool Stack (with license reality)

Layer	Tool	License / cost status
Network telemetry	Zeek, Suricata	Free, BSD/GPL, no change
Endpoint	Wazuh + Sysmon	Free, GPLv2, confirmed still fully free / no paywall
Vuln scanning	OpenVAS / Greenbone Community Edition	Free
Threat intel	MISP	Free, open source
Transport (deferred)	Apache Kafka (prod) or Redpanda Community (lab)	Kafka: Apache 2.0. Redpanda: BSL — free for self-hosted internal use, can't resell as a hosted service. Default to Kafka unless BSL terms are fine for your case.
Stream processing (deferred)	Apache Flink	Free, Apache 2.0 — add only once a single-node scorer can't keep up
Model serving	ONNX Runtime (in-process); Ollama/vLLM only for the SOC-copilot LLM	Free
Case management	DFIR-IRIS (default) or TheHive 5 Community	DFIR-IRIS: free, open, no use restriction. TheHive Community: free but license-gated to discovery/testing/training/education — review terms before using for real cases

Layer	Tool	License / cost status
Alert triage / AIOps	Keep (core workflow engine + HITL webhooks)	MIT / open source core; AI auto-correlation is Cloud/Enterprise-only — skip it, not needed
Active response	CrowdSec	Engine + community blocklist free (capped ~3k IPs unless you contribute signals)
Storage / Viz (deferred)	ClickHouse, Grafana	Free, Apache 2.0
Datasets	UNSW-NB15, CIC-IDS2017/2018, TON_IoT	UNSW-NB15 & CIC-IDS: free, unrestricted research/commercial use. TON_IoT: free for academic research only — commercial use needs UNSW's permission
Hosting (optional free cloud)	Oracle Cloud Always Free (Ampere A1)	Free but capped at 2 OCPU / 12GB total (cut from 4/24 in June 2026)

4. ML Evaluation Methodology (new)

Published >98–99% accuracy figures on UNSW-NB15/CIC-IDS2017 are well known in NIDS research to be partly inflated by non-behavioral dataset artifacts (near-constant TTLs, timestamp/flow-ID leakage that separates attack from benign traffic almost trivially). Treat those numbers as “the pipeline runs end to end,” not as a promise about real-world catch rate. The methodology below is what actually earns trust in the model:

- Split before you preprocess. Hold out data by time, attack scenario, or host — never a random row-level split. Fit encoders and scalers on the training split only, then apply them to validation/test; fitting on the full dataset first leaks information across the split.
- Report precision, recall, and PR-AUC — not raw accuracy. On real traffic, benign massively outnumbers malicious, so a model that always says “benign” still scores >95% accuracy while catching nothing.
- Track false positives per host-day, not just a percentage. This is the number that determines whether an analyst can actually work with the alert volume.
- Check calibration. A risk score of 0.85 should mean something consistent — verify the model isn't just confidently guessing.
- Validate cross-dataset. Train on one benchmark, test on another (or better, your own captured traffic) before trusting the numbers.
- Run in shadow mode first. The model scores live traffic and logs its output, but does not change alert severity or trigger any workflow, until its shadow-mode performance has been reviewed against real analyst judgment — typically several weeks of live traffic, not a single validation split.

5. Merged Phased Roadmap (revised)

Phase 0 — Scope & deployment tier (½–1 day)

- Local lab (16GB+ RAM) — best for learning and fast iteration.
- Oracle free-tier cloud (2 OCPU/12GB) — remote and always-on, but tight; skip Kafka/Flink at this size.
- Hybrid: train locally, deploy only the lightweight scoring agent to the free-tier VM.

Exit gate: you've written down which tier you're using and why — this decision determines whether later phases use Flink or a plain Python consumer.

Phase 1 — Foundational visibility (Week 1)

Deploy Wazuh manager + agents on all endpoints. This is the data source of truth for everything downstream.

Exit gate: Wazuh dashboard shows live events from every endpoint you intend to monitor.

Phase 2 — Enhanced endpoint telemetry (Week 2)

Add Sysmon on Windows endpoints (SwiftOnSecurity config as a starting ruleset). Confirm Wazuh is ingesting Sysmon events.

Exit gate: process-creation and network-connection events from Sysmon are visible in Wazuh, not just default Windows Event Log noise.

Phase 3 — Network visibility (Weeks 3–4)

Deploy Zeek (metadata-only, no alerting) and Suricata (ET Open rules, alerting) on a tap/SPAN port. Integrate both into Wazuh via custom decoders.

Exit gate: conn.log / dns.log and eve.json are flowing into Wazuh, and a manual nmap scan against a test host produces a visible Suricata alert.

Phase 4 — Proactive scanning (Week 5)

Stand up OpenVAS/Greenbone, run scheduled scans, correlate findings against asset inventory.

Exit gate: a scan has run against every asset in Phase 1's inventory and results are stored somewhere queryable.

Phase 5 — ML pipeline in shadow mode (Weeks 6–9)

- Preprocess UNSW-NB15/CIC-IDS2017/TON_IoT with a time/scenario-separated split (Section 4) — impute, encode, scale, SMOTE-Tomek for class imbalance, fit on train only.
- Train XGBoost/LightGBM + Isolation Forest, export to ONNX (skl2onnx).
- Score with a plain Python/ONNX consumer reading Wazuh's alert/archive API directly — no Kafka or Flink yet. Add them later only if a single-node scorer measurably can't keep up.
- Generate SHAP explanations from the model itself, attached to the risk score — never from the LLM.
- Run for several weeks in shadow mode: log risk scores and SHAP output, but don't let them change alert severity yet.

Exit gate: shadow-mode precision/recall and false-positives-per-host-day (Section 4) have been reviewed against analyst judgment on real traffic — only then does the model's score start affecting severity.

Phase 6 — Triage, case management, response (Weeks 10–11)

- Deploy Keep, connect it to Wazuh/ML alerts, build a workflow that enriches with MISP and routes high-risk alerts.

- Deploy DFIR-IRIS for case management (evaluate TheHive Community only if its discovery/testing/training/education license genuinely fits your use).
- If using a local LLM, restrict it to read-only summarization of existing SHAP output and case notes — it never writes to DFIR-IRIS, MISP, or the firewall directly.
- Connect CrowdSec bouncers, but gate every automated block behind Keep's HITL webhook approval — no auto-block without human sign-off.

Exit gate: a simulated attack (nmap + hydra against a test container) produces a Suricata alert, a Wazuh correlation, a shadow-mode ML score with SHAP, a Keep-routed alert, and a human-approved CrowdSec block — end to end.

Phase 7 — Scale-out, only if measured need justifies it (ongoing)

If the single-node scorer or Wazuh's own storage genuinely can't keep up with your event volume, add Kafka/Redpanda → Flink for streaming, and ClickHouse + Grafana for dashboards. Don't add these preemptively — they're operational overhead that only pays for itself at real scale.

6. Directory Structure

```
ids-cybersecurity-system/
|-- docker-compose.yml
|-- config/
|   |-- suricata/  zeek/  wazuh/  keep/
|-- ml_pipeline/
|   |-- datasets/  preprocess.py  train.py  export_onnx.py  models/
|   |-- eval/      # Section 4 methodology: splits, calibration, PR curves
|-- python_scorer/ # Phase 5 default; promote to flink_processor/ only if needed
|-- soc_copilot/
|   |-- app.py  prompt_templates.py # read-only summarizer, no write access
|-- dfir_iris/
|   |-- docker-compose.override.yml
|-- tests/
|   |-- replay/  attack-scenarios/ # tcpreplay + nmap/hydra scripted checks
```

7. Security Guardrails

Everything below stays regardless of build scale — it's what keeps a detection system from becoming its own liability.

- Don't auto-block anything until you trust the model. Keep every automated action behind Keep's HITL approval for the first several weeks of live traffic, minimum.
- Treat the model and LLM as attack surface. An attacker can shape flows to poison future training data, or place text in log fields (a User-Agent string, a DNS query) that a log-summarizing LLM later reads — that's a prompt-injection vector. Keep the LLM read-only and never let it trigger actions directly.
- Triton, if used at all (LLM/SOC-copilot layer only), has had real CVEs — pin recent versions and patch on a schedule. Since it's now off the hot path, exposure is much lower than a per-flow design would carry.
- Free tiers drift. Oracle just cut its ARM allocation; re-verify any “always free” cloud claim at deployment time rather than banking on it long-term.
- Benchmark accuracy ≠ real-world accuracy on these public datasets — Section 4's methodology exists precisely to catch that gap before it reaches production judgment.

- Hardware reality check: the full stack (Wazuh + Keep + DFIR-IRIS + MISP, plus Kafka/Flink/ClickHouse if you reach Phase 7) genuinely needs 16GB+ RAM to run comfortably.
- If this ever moves toward protecting real infrastructure: add a written threat model, pin and track component versions (a simple SBOM), and put a recurring review on the calendar — informal is fine for a lab, not for production.

8. Immediate Next Steps

- Confirm your deployment tier (Phase 0) — determines whether Phase 5 stays on a plain Python scorer or eventually needs Flink.
- Stand up Wazuh first and get it stable before touching anything else.
- When you reach Phase 5, build the eval/ split-and-metrics harness (Section 4) before you trust a single accuracy number.
- When you reach Phase 6, default to DFIR-IRIS — only consider TheHive Community if you've read its license and it genuinely fits.